Adaptive Load Balancer: Deliverable 1 code explained

Team 9 · Project 20 · every file in lb-d1 and what each block does

File	Block	What it does
client.py	**Imports**	**socket** (TCP), threading, json, argparse, Counter
	Options	Target 10.0.0.100:8080, 4 threads, 10 requests per thread, --name
	Counter + lock	**counts** tallies replies per server; **lock** keeps updates safe between threads
	one()	Opens a **new TCP connection**, sends {"id": n}, reads {"server": "srv2"}; a timeout or failure counts as "error"
	worker()	Each thread sends its requests in a loop and adds each reply to counts
	Threads	Starts all threads at once, then waits for them to finish
	Print	Prints h1: {"srv1": 9, "srv2": 15, "srv3": 16}
server.py	**Imports**	**socket** (TCP and UDP), threading, json, time, argparse
	Options	--name, --port 8080, --work, --report 10.0.0.254:9999, --interval 0.5
	active + lock	Counts connections being served right now, safely
	handle()	Per client: active +1, reads the request, does fake work, replies {"id", "server"}, closes, active −1; errors get {"error"}
	reporter()	Every 0.5 s sends a **UDP datagram** {"srv", "active"} to 10.0.0.254:9999
	main()	Starts the reporter, opens a TCP socket on port 8080, accepts clients and gives each one a thread
controller.py	**Settings**	Shared address 10.0.0.100:8080, report address 10.0.0.254:9999, s1 = switch 1, rules expire after 10 s idle, SERVERS (IP, MAC, s1 port 5 or 6)
	__init__	Memory: load, sent, mac_to_port, conns
	add_flow() / packet_out()	Helpers that install a rule or send a packet
	pick()	Picks the lowest **reported active + sent**, then adds 1 to its sent
	features()	When a switch connects: installs table-miss → controller and UDP reports → controller
	packet_in()	Sorts each packet to ARP reply, load report, new connection, drop, or learning switch
	arp_reply()	Answers ARP for 10.0.0.100 and 10.0.0.254
	on_report()	Saves the server's active count and resets its sent
	new_connection()	Reuses the server for a repeated SYN, else calls pick(); installs **forward and reverse NAT rules** on s1 and logs the choice
	learning_switch()	Normal switching: learns MAC → port, installs a rule or floods

File	Block	What it does
topo.py	**Settings**	h1–h3 clients, srv1–srv3 servers, srv1 slow, links 10 Mbit/s
	build()	s1, s2, s3; h1–h4 on s1; link A (s1:5 ↔ s2), link B (s1:6 ↔ s3); srv1 and srv2 on s2, srv3 on s3
	demo()	Starts the servers, pings, runs 120 requests from h1–h3 at once, checks srv1 got the fewest, prints **PASS or FAIL**
	main()	Cleans up, starts Ryu, builds the network, runs the demo, opens the CLI, stops everything
test_socket.py	**UDP listener**	Waits on port 19999 for a load report, standing in for the controller
	Start server	Runs server.py on 127.0.0.1:18080
	Test 1 (TCP)	4 threads × 5 requests, all 20 answered → PASS TCP
	Test 2 (UDP)	Report is valid JSON with the right fields → PASS UDP
	Cleanup	Always stops the server
README.md	**Documentation**	Objectives, topology, communication flow, how to run, test cases

In one line: the servers report their load over UDP, Ryu picks the least-loaded server for each new TCP connection and installs NAT rules on s1, topo.py builds the network and runs the demo, and test_socket.py checks the socket code on its own.